

Exercise P1. Lexical Scanner for Simplified Programming Language Ada

1 Aim of the Exercise

The aim of the exercise is to build a simple scanner for a much simplified version of the programming language Ada. The task of the analyzer is:

- to recognize tokens of C language and to determine their values
- to remove blanks and comments
- to recognize given directives
- to recognize lexical errors

2 Preliminaries

After turning on the computer, one should select Linux, and log in as *student*. One should open a console window (e.g. press **Alt-F2** and type **xterm**), create one's own directory using a command **mkdir** *family name of the user*, and a subdirectory for the current exercise. Download files for the C language from the Moodle web page of the course for the subject *Lexical Analysis*. The following files are to be found there:

- **p1a.pdf** — manual (just being read)
- **Makefile** — needed for compilation with the command **make**
- **ada.1** — skeletal lexical analyzer that needs to be completed; take a closer look at the definition of **process_token()**, which should be used in the rules
- **ada.y** — parser that is needed only for declaring tokens and for invoking the lexical analyzer
- **test1.adb** — correct test program
- **test2.adb** — incorrect test program

After having completed the exercise, the directory should be removed.

3 Tasks

The supplied skeletal lexical analyzer should be extended so that it works correctly on supplied test programs. The analyzer should print information on recognized tokens in three columns:

1. matched text
2. recognized token
3. value of the token (only when it makes sense)

Function **process_token** is designed to print that information. The function returns a recognized token, so an action in a rule recognizing a token should contain **return process_token(. . .)** with appropriate parameters.

The supplied code needs to be completed with the following items:

- A. printing one's own name (in the **bison** program)
- B. detecting keywords in test programs (defined in the source program for **bison**)
- C. removing blanks
- D. removing one-line comments (without start conditions)
- E. recognition of multi-character operators (**<=**, **=>**, ...) that appear in test programs
- F. recognition of identifiers
- G. recognition of integers

- H. recognition of real numbers
- I. recognition of strings without start conditions
- J. recognition of one-character tokens: operators and punctuation
- K. recognition of character constants
- L. recognition of strings using start conditions
- M. ignoring comments using start conditions (yes, this is artificial)
- N. recognition of strings not closed in one line using start conditions (end-of-line in a string)
- O. recognition of strings containing double quotes (written as two double quotes one after another) using start conditions

4 Grading

All items are graded as 1 point. If needed, items from K to O can be completed **at home for half a point each**. The file developed in the lab should be uploaded before the end of the class on Moodle. **The lexical analyzer will be needed for the next exercise.**

5 Start Conditions

- Start condition active at the start of the program: INITIAL
- Declaration: %x condition1, condition2, . . .
- Matching in a start condition:


```
<con1> re1      action1;
<con1,con2,INITIAL>re2 action2;
<*>re3         działanie3
```
- changing start condition: BEGIN condition4
- current start condition: YYSTATE
- checking the current start condition after all input data has been read: in function yywrap, which must be defined, and which must return 1

6 Test Data — File test1.adb

```

1  — Ten program drukuje znaki i ich kody ASCII
2  — Kompilacja: gnatmake listascii
3
4  with Ada.Text_IO; use Ada.Text_IO;
5  with Ada.Integer_Text_IO;
6
7  procedure ListAscii is
8    FromASCII : constant Integer := 32;
9    ToASCII   : constant Integer := 127;
10   Uc : Character := ' ';           — zmienna
11   Fl : Float;
12   T  : array(1..10) of Float;
13   I  : Integer;
14   D  : record
15     Dzien, Miesiac : Integer;
16     Rok : Integer;
17   end record;
18 begin
19   New_Line;
20   Put_Line("Kody ASCII:"); Put_Line("=====");
21   for I in Integer range FromASCII .. ToASCII loop

```

```

22     Ada.Integer_Text_IO.Put(I);  — procedura z Ada.Integer_Text_IO
23     Put(' ');  — to jest procedura z Ada.Text_IO widoczna z powodu ,,use''
24     Put(Uc);
25     Uc := Character'Succ(Uc);  — kolejny znak
26     New_Line;
27 end loop;
28 — tylko test
29 Fl := 1.1 * 0.1 + 1.0e-2;  — w rzeczywistych po kropce i przed zawsze cyfra
30 T(0) := 0;
31 for I in 1 .. 10 loop
32     T(I) := T(I) * I * I;
33 end loop;
34 D.Dzien := 1;
35 D.Miesiac := D. Dzien * 10;
36 D.Rok := 2018;
37 end ListAscii;

```

7 Test Data – File test2.adb

```

1  — Ten program drukuje znaki i ich kody ASCII
2  — Kompilacja: gnatmake listascii
3
4  with Ada.Text_IO; use Ada.Text_IO;
5  with Ada.Integer_Text_IO;
6
7  procedure ListAscii is
8      FromASCII : constant Integer := 32;
9      ToASCII : constant Integer := 127;
10     Uc : Character := ' ';  — zmienna
11     Fl : Float;
12     T : array(1..10) of Float;
13     I : Integer;
14     D : record
15         Dzien, Miesiac : Integer;
16         Rok : Integer;
17     end record;
18 begin
19     New_Line;
20     Put_Line("Kody ASCII:"); Put_Line("=====");
21     for I in Integer range FromASCII .. ToASCII loop
22         Ada.Integer_Text_IO.Put(I);  — procedura z Ada.Integer_Text_IO
23         Put(' ');  — to jest procedura z Ada.Text_IO widoczna z powodu ,,use''
24         Put(Uc);
25         Uc := Character'Succ(Uc);  — kolejny znak
26         New_Line;
27     end loop;
28     — tylko test
29     Fl := 1.1 * 0.1 + 1.0e-2;  — w rzeczywistych po kropce i przed zawsze cyfra
30     T(0) := 0;
31     for I in 1 .. 10 loop
32         T(I) := T(I) * I * I;
33     end loop;
34     D.Dzien := 1;
35     D.Miesiac := D. Dzien * 10;
36     D.Rok := 2018;
37 end ListAscii;

```

8 Output of the Lexical Analyzer for test1.adb

1	Imie i Nazwisko		
2	yytext	Typ tokena	Wartosc tokena znakowo

3			
4	with	KW.WITH	
5	Ada	IDENT	Ada
6	.	.	
7	Text_IO	IDENT	Text_IO
8	;	;	
9	use	KW.USE	
10	Ada	IDENT	Ada
11	.	.	
12	Text_IO	IDENT	Text_IO
13	;	;	
14	with	KW.WITH	
15	Ada	IDENT	Ada
16	.	.	
17	Integer.Text_IO	IDENT	Integer.Text_IO
18	;	;	
19	procedure	KW.PROCEDURE	
20	ListAscii	IDENT	ListAscii
21	is	KW.IS	
22	FromASCII	IDENT	FromASCII
23	:	:	
24	constant	KW.CONSTANT	
25	Integer	IDENT	Integer
26	:=	ASSIGN	
27	32	INTEGER.CONST	32
28	;	;	
29	ToASCII	IDENT	ToASCII
30	:	:	
31	constant	KW.CONSTANT	
32	Integer	IDENT	Integer
33	:=	ASSIGN	
34	127	INTEGER.CONST	127
35	;	;	
36	Uc	IDENT	Uc
37	:	:	
38	Character	IDENT	Character
39	:=	ASSIGN	
40	' ,	CHARACTER.CONST	' ,
41	;	;	
42	F1	IDENT	F1
43	:	:	
44	Float	IDENT	Float
45	;	;	
46	T	IDENT	T
47	:	:	
48	array	KW.ARRAY	
49	(	(	
50	1	INTEGER.CONST	1
51	..	RANGE	
52	10	INTEGER.CONST	10
53	)	)	
54	of	KW.OF	
55	Float	IDENT	Float
56	;	;	
57	I	IDENT	I
58	:	:	
59	Integer	IDENT	Integer
60	;	;	
61	D	IDENT	D
62	:	:	
63	record	KW.RECORD	
64	Dzien	IDENT	Dzien
65	,	,	
66	Miesiac	IDENT	Miesiac
67	:	:	
68	Integer	IDENT	Integer

69	;	;	
70	Rok	IDENT	Rok
71	:	:	
72	Integer	IDENT	Integer
73	;	;	
74	end	KW.END	
75	record	KW.RECORD	
76	;	;	
77	begin	KW.BEGIN	
78	New_Line	IDENT	New_Line
79	;	;	
80	Put_Line	IDENT	Put_Line
81	(	(	
82	"Kody ASCII:"	STRING.CONST	"Kody ASCII:"
83	)	)	
84	;	;	
85	Put_Line	IDENT	Put_Line
86	(	(	
87	"=====	STRING.CONST	"=====
88	" "	STRING.CONST	" "
89	)	)	
90	;	;	
91	for	KW.FOR	
92	I	IDENT	I
93	in	KW.IN	
94	Integer	IDENT	Integer
95	range	KW.RANGE	
96	FromASCII	IDENT	FromASCII
97	..	RANGE	
98	ToASCII	IDENT	ToASCII
99	loop	KW.LOOP	
100	Ada	IDENT	Ada
101	.	.	
102	Integer_Text_IO	IDENT	Integer_Text_IO
103	.	.	
104	Put	IDENT	Put
105	(	(	
106	I	IDENT	I
107	)	)	
108	;	;	
109	Put	IDENT	Put
110	(	(	
111	' ,	CHARACTER.CONST	' ,
112	)	)	
113	;	;	
114	Put	IDENT	Put
115	(	(	
116	Uc	IDENT	Uc
117	)	)	
118	;	;	
119	Uc	IDENT	Uc
120	:=	ASSIGN	
121	Character	IDENT	Character
122	,	,	
123	Succ	IDENT	Succ
124	(	(	
125	Uc	IDENT	Uc
126	)	)	
127	;	;	
128	New_Line	IDENT	New_Line
129	;	;	
130	end	KW.END	
131	loop	KW.LOOP	
132	;	;	
133	F1	IDENT	F1
134	:=	ASSIGN	

135	1.1	FLOAT.CONST	1.1
136	*	*	
137	0.1	FLOAT.CONST	0.1
138	+	+	
139	1.0e-2	FLOAT.CONST	1.0e-2
140	;	;	
141	T	IDENT	T
142	(	(	
143	0	INTEGER.CONST	0
144	)	)	
145	:=	ASSIGN	
146	0	INTEGER.CONST	0
147	;	;	
148	for	KWFOR	
149	I	IDENT	I
150	in	KW_IN	
151	1	INTEGER.CONST	1
152	..	RANGE	
153	10	INTEGER.CONST	10
154	loop	KWLOOP	
155	T	IDENT	T
156	(	(	
157	I	IDENT	I
158	)	)	
159	:=	ASSIGN	
160	T	IDENT	T
161	(	(	
162	I	IDENT	I
163	)	)	
164	*	*	
165	I	IDENT	I
166	*	*	
167	I	IDENT	I
168	;	;	
169	end	KWEND	
170	loop	KWLOOP	
171	;	;	
172	D	IDENT	D
173	.	.	
174	Dzien	IDENT	Dzien
175	:=	ASSIGN	
176	1	INTEGER.CONST	1
177	;	;	
178	D	IDENT	D
179	.	.	
180	Miesiac	IDENT	Miesiac
181	:=	ASSIGN	
182	D	IDENT	D
183	.	.	
184	Dzien	IDENT	Dzien
185	*	*	
186	10	INTEGER.CONST	10
187	;	;	
188	D	IDENT	D
189	.	.	
190	Rok	IDENT	Rok
191	:=	ASSIGN	
192	2018	INTEGER.CONST	2018
193	;	;	
194	end	KWEND	
195	ListAscii	IDENT	ListAscii
196	;	;	

9 Output of the Lexical Analyzer for test2.adb

1	Imie i Nazwisko		
2	yytext	Typ tokena	Wartosc tokena znakowo
3			
4	with	KW.WITH	
5	Ada	IDENT	Ada
6	.	.	
7	Text_IO	IDENT	Text_IO
8	;	;	
9	use	KW.USE	
10	Ada	IDENT	Ada
11	.	.	
12	Text_IO	IDENT	Text_IO
13	;	;	
14	with	KW.WITH	
15	Ada	IDENT	Ada
16	.	.	
17	Integer_Text_IO	IDENT	Integer_Text_IO
18	;	;	
19	procedure	KW.PROCEDURE	
20	ListAscii	IDENT	ListAscii
21	is	KW.IS	
22	FromASCII	IDENT	FromASCII
23	:	:	
24	constant	KW.CONSTANT	
25	Integer	IDENT	Integer
26	:=	ASSIGN	
27	32	INTEGER.CONST	32
28	;	;	
29	ToASCII	IDENT	ToASCII
30	:	:	
31	constant	KW.CONSTANT	
32	Integer	IDENT	Integer
33	:=	ASSIGN	
34	127	INTEGER.CONST	127
35	;	;	
36	Uc	IDENT	Uc
37	:	:	
38	Character	IDENT	Character
39	:=	ASSIGN	
40	' '	CHARACTER.CONST	' '
41	;	;	
42	F1	IDENT	F1
43	:	:	
44	Float	IDENT	Float
45	;	;	
46	T	IDENT	T
47	:	:	
48	array	KW.ARRAY	
49	(	(	
50	1	INTEGER.CONST	1
51	..	RANGE	
52	10	INTEGER.CONST	10
53	)	)	
54	of	KW.OF	
55	Float	IDENT	Float
56	;	;	
57	I	IDENT	I
58	:	:	
59	Integer	IDENT	Integer
60	;	;	
61	D	IDENT	D
62	:	:	
63	record	KW.RECORD	
64	Dzien	IDENT	Dzien

65	,	,	
66	Miesiac	IDENT	Miesiac
67	:	:	
68	Integer	IDENT	Integer
69	;	;	
70	Rok	IDENT	Rok
71	:	:	
72	Integer	IDENT	Integer
73	;	;	
74	end	KW.END	
75	record	KW.RECORD	
76	;	;	
77	begin	KW.BEGIN	
78	New_Line	IDENT	New_Line
79	;	;	
80	Put_Line	IDENT	Put_Line
81	(	(	
82	"Kody ASCII:"	STRING.CONST	"Kody ASCII:"
83	)	)	
84	;	;	
85	Put_Line	IDENT	Put_Line
86	(	(	
87	"=====	STRING.CONST	"=====
88	New line in a string constant!		
89	for	KW.FOR	
90	I	IDENT	I
91	in	KW.IN	
92	Integer	IDENT	Integer
93	range	KW.RANGE	
94	FromASCII	IDENT	FromASCII
95	..	RANGE	
96	ToASCII	IDENT	ToASCII
97	loop	KW.LOOP	
98	Ada	IDENT	Ada
99	.	.	
100	Integer_Text_IO	IDENT	Integer_Text_IO
101	.	.	
102	Put	IDENT	Put
103	(	(	
104	I	IDENT	I
105	)	)	
106	;	;	
107	Put	IDENT	Put
108	(	(	
109	' ,	CHARACTER.CONST	' ,
110	)	)	
111	;	;	
112	Put	IDENT	Put
113	(	(	
114	Uc	IDENT	Uc
115	)	)	
116	;	;	
117	Uc	IDENT	Uc
118	:=	ASSIGN	
119	Character	IDENT	Character
120	,	,	
121	Succ	IDENT	Succ
122	(	(	
123	Uc	IDENT	Uc
124	)	)	
125	;	;	
126	New_Line	IDENT	New_Line
127	;	;	
128	end	KW.END	
129	loop	KW.LOOP	
130	;	;	

131	F1	IDENT	F1
132	:=	ASSIGN	
133	1.1	FLOAT.CONST	1.1
134	*	*	
135	0.1	FLOAT.CONST	0.1
136	+	+	
137	1.0e-2	FLOAT.CONST	1.0e-2
138	;	;	
139	T	IDENT	T
140	(	(	
141	0	INTEGER.CONST	0
142	)	)	
143	:=	ASSIGN	
144	0	INTEGER.CONST	0
145	;	;	
146	for	KWFOR	
147	I	IDENT	I
148	in	KW.IN	
149	1	INTEGER.CONST	1
150	..	RANGE	
151	10	INTEGER.CONST	10
152	loop	KWLOOP	
153	T	IDENT	T
154	(	(	
155	I	IDENT	I
156	)	)	
157	:=	ASSIGN	
158	T	IDENT	T
159	(	(	
160	I	IDENT	I
161	)	)	
162	*	*	
163	I	IDENT	I
164	*	*	
165	I	IDENT	I
166	;	;	
167	end	KW.END	
168	loop	KWLOOP	
169	;	;	
170	D	IDENT	D
171	.	.	
172	Dzien	IDENT	Dzien
173	:=	ASSIGN	
174	1	INTEGER.CONST	1
175	;	;	
176	D	IDENT	D
177	.	.	
178	Miesiac	IDENT	Miesiac
179	:=	ASSIGN	
180	D	IDENT	D
181	.	.	
182	Dzien	IDENT	Dzien
183	*	*	
184	10	INTEGER.CONST	10
185	;	;	
186	D	IDENT	D
187	.	.	
188	Rok	IDENT	Rok
189	:=	ASSIGN	
190	2018	INTEGER.CONST	2018
191	;	;	
192	end	KW.END	
193	ListAscii	IDENT	ListAscii
194	;	;	